裁判测试端使用教程v1.1

更新:
选手测试裁判使用教程
一、准备环境与安装
二、先用参考车确认环境
三、测试自己的第一阶段程序
1. 启动测试裁判
2. 配置自己的程序
3. 开始并确认双向通信
4. 第一阶段通过标准
四、测试自己的第二阶段程序
1. 切换阶段并设置陪练
2. 开始并观察行为
五、测试 LLM 调用
1. 模拟模式：验证请求链路
2. 真实模式：验证模型理解和策略
六、网页操作说明
七、异机测试
八、常见问题
九、上场前检查
十、提交前置检录材料

更新:

- **第四部分「测试自己的第二阶段程序」**：同步500点满分门槛，加入计分公式、死亡扣60、击杀平分60及任务保留说明。
- **同一部分**：补充普通攻击、反伤、辣椒及其他环境死亡的奖励区别，说明不能重复获得击杀奖励。
- **第十部分「提交前置检录材料」**：补充提交自己的程序完成感知、Tick/场次回填、动作及回执验证的截图或录屏；截止时间为 **10月31日20:00**，参考车记录不能替代，材料不得包含密钥或正式令牌。

- 另整理了解压目录、真实 AI 重定向提示及故障排查措辞。启动命令、话题名称和测试令牌未因本次计分调整改变。

选手测试裁判使用教程

本教程用于验证参赛程序能否完成第一阶段资源采集、第二阶段对抗与任务，以及 LLM 接入。测试地图和事件均为独立练习数据，不代表正式比赛场景。

测试通过后可使用同一套参赛程序上场，按主办方通知更换席位、ROS 网络配置、AI 网关地址及令牌，并完成现场联调。不要在程序中写死测试令牌、场次 ID、地图或事件时间。

一、准备环境与安装

运行环境：Ubuntu 22.04、ROS 2 Humble、x86_64。使用虚拟机时，以下命令在 Ubuntu 虚拟机内执行。建议先在同一 Ubuntu 环境中运行测试裁判和自己的程序，确认通过后再测试异机通信。

收到以下两个文件后，将它们放在同一个目录：

- `rmtc_practice.tar.gz`
- `rmtc_practice.sha256`

已安装 ROS 2 Humble 的设备执行：

```
sudo apt install ros-humble-rclpy ros-humble-std-msgs \
  ros-humble-rmw-fastrtps-cpp python3-requests libcurl4

sha256sum -c rmtc_practice.sha256
tar xzf rmtc_practice.tar.gz
cd rmtc_practice
```

校验结果应显示 `rmtc_practice.tar.gz: OK` 或“成功”。解压后无需构建裁判程序。

若主办方另行提供已经解压的目录，按其说明进入。后续启动命令均在测试包目录执行。

二、先用参考车确认环境

这一项用于验证安装和通信环境，不代表自己的参赛程序已经通过测试。

终端 1：启动第一阶段测试裁判。

```
./start_referee.sh --stage 1
```

打开浏览器 http://127.0.0.1:8080。等待页面显示“暂停”。第一阶段默认只有 A 席位。

终端 2：进入同一测试包目录，启动参考车。

```
./start_bot.sh A '环境验证车'
```

在网页点击“开始 / 继续”。确认 Tick 推进、A 车运动，右侧“动作 Tick”和“执行 Tick”出现数值。

验证后，在终端 2 按 `Ctrl+C` 停止参考车。在网页点击“按原配置重开”，再接入自己的程序。**自己的程序与参考车不能同时控制 A 席位。**

三、测试自己的第一阶段程序

1. 启动测试裁判

如果上一节的裁判还在运行，可直接使用；否则执行：

```
./start_referee.sh --stage 1
```

默认第一阶段为 **100 ms/Tick、100 Tick**，不计暂停时长共 10 秒。初次调试希望观察得更慢时，可在网页调整“Tick 毫秒”和“总 Tick”，然后点击“部署新练习”。最终应恢复默认节奏验证。

2. 配置自己的程序

在启动自己程序的终端执行：

```
source /opt/ros/humble/setup.bash
export ROS_DOMAIN_ID=27
export ROS_LOCALHOST_ONLY=0
export RMW_IMPLEMENTATION=rmw_fastrtps_cpp
export RMTC_TEAM_TOKEN=practice-A
```

再加载自己工作空间的 `install/setup.bash`，按自己项目的启动方式运行程序。程序须使用以下接入信息：

项目	A 席位配置
席位	`A`
感知订阅	`/team_A/perception`
动作发布	`/team_A/action`

可选显示名称注册	`/team_A/registration`
ROS 消息类型	`std_msgs/msg/String`
本机 AI 请求地址	`http://127.0.0.1:8080/v1/chat/completions`
测试 AI 令牌	`practice-A`

环境变量不会自动配置任意参赛程序。须确认自己的代码读取了相应变量，或通过配置文件、启动参数传入了这些值。

3. 开始并确认双向通信

点击网页“开始 / 继续”，同时查看自己程序的日志：

1. 能持续解析当前感知，并取得 `tick` 、 `session_id` 和 `self.position` 。
2. 每个新 Tick 发布至多一个动作，原样回填该帧的 `tick` 和 `session_id` 。
3. 后续感知的 `self.last_action` 能确认上一 Tick 动作结果。
4. 网页右侧的“动作 Tick”和“执行 Tick”持续更新。

动作示例（JSON 是 ROS 消息 `data` 的内容）：

```
{"tick": 12, "session_id": "practice-示例", "action": "move", "direction": "right"}
```

示例中的 Tick 和场次 ID 必须换成程序当前收到的值，不能直接重复发送示例。不要用手工复制感知后再输入动作的方式验证正常控制，在较短 Tick 下动作可能已经过期。

本车 `self.position` 为绝对坐标；视野实体的位置格式按随包 `CONTESTANT_PROTOCOL.md` 处理，不要把视野位置直接当成地图绝对坐标。

4. 第一阶段通过标准

- 程序在默认节奏下持续接收感知、发布动作、处理回执。
- 可以自主移动和采集，分数或资源统计随采集变化。
- 重开后使用新感知中的 `session_id` ，不继续发送上一场的动作。
- 临时停止并重启选手程序后，可以根据当前感知重新定位并继续运行。

四、测试自己的第二阶段程序

1. 切换阶段并设置陪练

可在网页将“阶段”切换到“第二阶段 · 对抗与任务”，勾选 B、C、D 为陪练，然后点击“部署新练习”。A 留给自己的程序。

也可先在裁判终端按 `Ctrl+C` 退出，再启动：

```
./start_referee.sh --stage 2 --bots B,C,D
```

默认第二阶段为 **500 ms/Tick、500 Tick**，不计暂停时长共 4 分 10 秒。最多添加三台陪练，也可以不添加。未接入的车辆等待，不会阻止比赛推进。

如果四支真实程序同时测试，每个程序分别使用 A/B/C/D 中不同席位，关闭相应陪练。修改席位时，感知、动作、注册话题和测试令牌必须一起修改，例如 B 使用 `/team_B/perception` 、 `/team_B/action` 和 `practice-B` 。

2. 开始并观察行为

自己的程序接入后，点击“开始 / 继续”。重点验证：

- 移动、攻击、防御及等待动作能正确执行。
- 能处理受伤、阵亡、复活及位置变化。
- 能读取自然语言任务和随机事件公告。
- 能依据 LLM 返回结果调整策略，且 LLM 请求等待期间不会阻塞感知处理和逐 Tick 动作。

每 Tick 先按移动前位置结算攻击伤害及死亡，再执行存活车辆移动。被击毁的车辆不能执行该 Tick 的移动。每 Tick 的计分伤害量为 `max(0, 上一 Tick 累计值 + 本 Tick 实际攻击伤害 + 击杀奖励 − 60 × 本 Tick 死亡次数)` 。累计值不截断，达到 500 时对抗项取得满分 15 分；同分排名及 `effective_damage` 伤害任务使用此值。已完成任务不因随后死亡扣除而撤销。

普通攻击及反伤按致死战斗批次中实际造成正伤害的贡献者去重，平分每名死者的 60 点击杀奖励；此前 Tick 的攻击者不参与分配。同 Tick 的辣椒火线统一结算，实际造成正伤害的触发者去重后平分 60 点奖励，已在战斗阶段阵亡的车辆不重复产生辣椒击杀奖励。爆炸、世界分裂等环境死亡扣除 60 点，不发放击杀奖励。辣椒及反伤的损血不额外计入普通攻击贡献，其击杀奖励单独计入。

网页中的全图、配置和本地日志用于人工调试。参赛程序只能使用通信协议允许的接口，不能从网页或本地文件读取全图、事件参数作为决策输入。

五、测试 LLM 调用

1. 模拟模式：验证请求链路

默认使用模拟模式，无需模型 API Key。它返回固定响应，不会理解地图、任务或随机事件。

A 席位请求示例：

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer practice-A' \
  -d '{"messages":[{"role":"user","content":"请给出一个测试建议"}],"stream":false}'
```

若系统没有 `curl`，可安装 `curl` 后执行，或直接使用自己的 HTTP 客户端发起同样请求。

页面应显示“模拟成功 1 次”。**手工请求也会增加次数，因此验证自己程序前应重开场次，再只让自己的程序调用。**

模拟成功只代表 HTTP 链路可用，不能证明真实模型调用或事件理解成功。

2. 真实模式：验证模型理解和策略

先退出正在运行的测试裁判。在裁判终端设置自己的 OpenAI 兼容模型服务：

```
export PRACTICE_LLM_URL='https://你的服务/v1/chat/completions'
read -rsp '请输入自己的模型 API Key: ' PRACTICE_LLM_KEY; echo
export PRACTICE_LLM_KEY
export PRACTICE_MODEL='你的模型名称'

./start_referee.sh --stage 2 --ai-mode real
```

`PRACTICE_LLM_URL` 须填写服务商提供的**完整聊天请求地址**。示例地址及模型名称须替换为真实配置。代理不跟随重定向；如提示上游重定向，须改填服务的最终请求地址。

自己的参赛程序访问本地 `http://127.0.0.1:8080/v1/chat/completions`，携带 `Authorization: Bearer practice-A`。如果请求体包含 `model`，它优先于 `PRACTICE_MODEL`；如果省略 `model`，使用裁判启动终端设置的模型名称。仅支持非流式请求，`stream` 应省略或设为 `false`。

真实模式下，页面显示“LLM 成功 N 次”。计数按本场、本席位累计，失败不增加成功次数，重开后归零。次数表示请求成功，不代表模型策略正确；还要查看自己的模型输出、解析结果及实际动作。

真实调用消耗自己的模型额度。若加入陪练，陪练也可能调用模型。建议先只接自己的程序，验证成功后再添加陪练。

随机事件应将公开自然语言公告交给真实 LLM 理解。不要将练习配置中的结构化事件参数或日程写入策略。

六、网页操作说明

控件或区域	使用方式
阶段	选择第一或第二阶段；会重新载入该阶段练习模板
Tick 毫秒、总 Tick	修改下一场速度和长度，点击“部署新练习”后生效
陪练	勾选由测试端控制的席位，不勾自己的席位
部署新练习	使用当前表单创建新场次，部署后暂停，场次 ID 更新
开始 / 继续	开始或恢复 Tick 推进
暂停	暂停 Tick 推进，便于查看日志和状态
按原配置重开	按已部署配置创建新场次，不采用尚未部署的表单修改
地图、任务与事件配置	展开后可编辑 JSON，或导入、导出练习配置；修改只用于下一次部署
接入诊断	查看感知、动作、执行 Tick 和调用次数
动作回执 / 拒绝原因	查看公开感知回执及接入错误
公开事件公告	查看当前公开文本，不能用页面替代参赛程序订阅

编辑配置后建议先“导出配置”留存，再部署。错误配置不会替换当前场次；刷新页面会丢失尚未保存的编辑。切换阶段也会载入模板，切换前先保存自定义配置。

七、异机测试

先确认本机测试通过，再将裁判和选手程序分开运行。

两台设备的 ROS 终端使用相同配置：

```
source /opt/ros/humble/setup.bash
export ROS_DOMAIN_ID=27
export ROS_LOCALHOST_ONLY=0
export RMW_IMPLEMENTATION=rmw_fastrtps_cpp
```

两台设备须位于能够进行 ROS 双向通信的局域网。域编号相同不等于网络一定可用，必须验证实际感知和动作回执。虚拟机应具有可与另一台设备通信的网络配置；仅能 SSH 登录虚拟机不能证明 ROS 通信正常。

测试端网页和 AI 代理默认只供本机访问。异机访问时，在**选手设备**建立 SSH 转发：

```
ssh -N -L 8080:127.0.0.1:8080 用户名@测试裁判机IP
```

保持这个终端运行。在选手设备打开 http://127.0.0.1:8080，自己的程序也使用本地 `http://127.0.0.1:8080/v1/chat/completions` 。

若选手设备的 8080 端口被占用，改为：

```
ssh -N -L 18080:127.0.0.1:8080 用户名@测试裁判机IP
```

随后将浏览器地址和自己的 AI 网关地址改为本地 18080 端口。**SSH 转发只处理网页及 AI HTTP 请求，不转发 ROS 话题。**

八、常见问题

现象	检查方法
网页打不开	检查裁判终端是否显示就绪、端口是否正确；虚拟机中的 127.0.0.1 指虚拟机本身
话题存在，但没有感知	确认已点击开始、比赛未结束、席位正确，并检查 QoS、ROS 域和双向网络
网页有“感知 Tick”，自己的程序却收不到	该数值仅证明测试端看到发布；检查自己程序的订阅配置和日志
有“动作 Tick”，没有“执行 Tick”	检查动作的场次、Tick、类型和方向；结合后续 `self.last_action` 判断是否被接受
出现 `session_mismatch`	停止使用缓存场次 ID，从新感知更新 `session_id` 后发送动作

动作过期或不稳定	只处理最新 Tick，不批量补发历史动作；避免同步等待 LLM 阻塞控制循环
AI 返回 401 / `invalid_token`	测试使用与席位对应的 `practice-A/B/C/D`，确认 Bearer 请求头和访问地址
AI 返回 404	检查本地请求路径是否为 `/v1/chat/completions`；真实模式检查上游完整地址
AI 返回 429	本席位已有请求进行中，避免并发请求同一席位；检查程序调用频率
模拟成功，但车没有合理处理事件	模拟模式不理解事件，改用真实 LLM，并检查自己的提示词和输出解析
AI 网关报错但车辆能正常移动	动作通信与 AI 请求分别检查，不能仅凭车辆移动认定 AI 联调通过
重新部署后程序不工作	程序须接受新场次 ID、当前 Tick 和当前位置；清理上一场缓存
车辆行为混乱	检查是否有两个程序或陪练同时使用同一席位

观察 A 席位感知可使用：

```
ros2 topic echo /team_A/perception std_msgs/msg/String \
  --qos-reliability reliable --qos-durability volatile
```

在比赛运行时应持续收到消息。完整字段含义、QoS 和错误码以随包 `CONTESTANT_PROTOCOL.md` 为准。

日志目录显示在页面“本场日志”处，默认位于 `~/.rmtc-practice/`。排查时优先查看自己的程序日志，以及本场 `communication.jsonl` 和 `ai.jsonl`；反馈问题时附上阶段、席位、Tick、错误内容和复现步骤，不要附模型密钥。

九、上场前检查

- 两个阶段均已在默认 Tick 节奏下运行通过。
- 收到感知、提交动作、收到回执三个环节均已验证。
- 第二阶段已经使用真实 LLM 验证任务和事件处理，而非只验证模拟接口。
- 暂停恢复、重开、选手进程重启后均能恢复正常控制。

- 程序不读取网页、全图或测试配置作为策略输入。
- 上场时更换主办方提供的席位、ROS 网络配置、AI 网关地址和本场令牌。
- 使用实际比赛设备和有线网络完成一次双向联调。

十、提交前置检录材料

按赛事规则，将自己的程序完成感知接收、正确回填 tick/session_id、提交合法动作及处理执行回执的截图或录屏，与其他参赛材料一同于 **2026 年 10 月 31 日 20:00 前**提交。参考车运行记录不代替自己的程序记录。材料中不要包含模型密钥或正式令牌；具体提交要求以赛事规则及官方公告为准。